

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Северный (Арктический) федеральный университет имени М.В. Ломоносова»

Высшая школа информационных технологий и автоматизированных систем
(наименование высшей школы / филиала / института / колледжа)

ОТЧЕТ
о лабораторном практикуме

По дисциплине DevTestOps

На тему Основные возможности библиотеки Selenium в Python

Выполнил обучающийся:

Груздев Николай Александрович
(Ф.И.О.)

Направление подготовки / специальность:

09.03.01 Информатика и вычислительная
техника
(код и наименование)

Курс: 4

Группа: 151220

Руководитель: Тарасов А.П., старший
преподаватель кафедры информационных
систем и информационной безопасности САФУ
(Ф.И.О. руководителя, должность / уч. степень / звание)

Отметка о зачете

(отметка прописью)

(дата)

Руководитель

(подпись руководителя)

А.П. Тарасов
(инициалы, фамилия)

Архангельск 2025

ЛИСТ ДЛЯ ЗАМЕЧАНИЙ

ЦЕЛЬ РАБОТЫ

Получить практические навыки по работе с Senium.

1 РЕШЕНИЕ ПОСТАВЛЕННОЙ ЗАДАЧИ

Используя Selenium, проведём автоматизацию тестирования страницы авторизации, ранее созданной для Cypress. Для этого внесём изменения в HTML-код страниц Apache2. На странице входа добавим ID для кнопки, а на главной странице Apache2 — ID для первого текстового элемента (рис. 2).

В файле auth-apache2.html добавляем новые строки (листинг 1).

Листинг 1 – auth-apache2.html

```
<div>
    <button id="sign_in" type="button" onclick="sign_in()">Sign
    in</button>
</div>
```

```
<body>
  <div class="main_page">
    <div class="page_header floating_element">
      
      <span id="apache2" class="floating_element">
        Apache2 Debian Default Page
      </span>
    </div>
  </div>
</body>
```

Рисунок 1 – Правки в файле index.html

Далее напишем скрипт, который будет автоматически открывать страницу входа Apache2, вводить учётные данные и проверять успешный переход на целевую страницу (рис. 2).

```

from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Firefox()

# Подключение к странице Apache
driver.get("http://vm1:82")
time.sleep(2)

# Получение элементов HTML
login_field = driver.find_element(by=By.ID, value="login")
password_field = driver.find_element(by=By.ID, value="password")
sign_in_button = driver.find_element(by=By.ID, value="sign_in")

# Выполнение авторизации
login_field.send_keys("user")
time.sleep(2)
password_field.send_keys("1111")
time.sleep(2)
sign_in_button.click()

# Проверка открытия страницы Apache2
apache_page_message = driver.find_element(by=By.ID, value="apache2")
print(f"Успешно выполнен вход на страницу {apache_page_message.text}")
time.sleep(2)
driver.quit()

```

Рисунок 2 – Код скрипта test.py

При запуске скрипта Selenium откроет веб-браузер, загрузит страницу авторизации и заполнит поля логина и пароля (рис. 3).

Login page

Enter login

Enter password

This connection is not secure. Logins entered here could be compromised.
[Learn More](#)

Рисунок 3 – Авторизация

После этого произойдёт автоматический переход на главную страницу Apache2 (рис. 4).

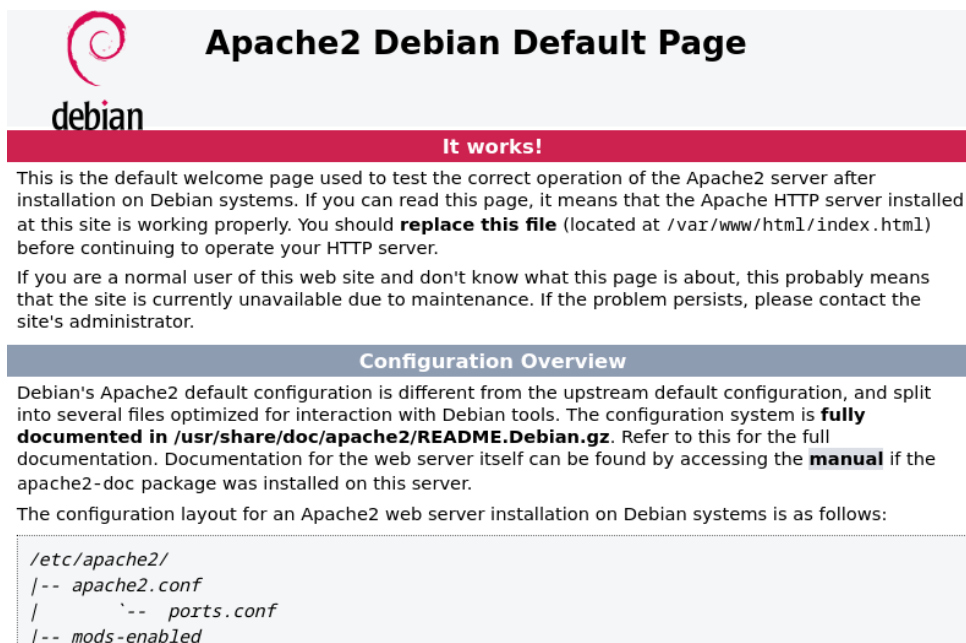


Рисунок 4 – Целевая страница Apache2

По завершении теста в консоли будет выведено сообщение об успешном входе в систему (рис. 5).

```
(venv) user@vm1:~/selenium$ python test.py  
Успешно выполнен вход на страницу Apache2 Debian Default Page
```

Рисунок 5 – Результат выполнения скрипта

2 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Неявные ожидания (Implicit Waits) — это глобальная настройка драйвера, которая задаёт единый временной лимит (например, 10 секунд) для поиска любого элемента на странице. Если элемент не найден в течение этого времени, драйвер генерирует исключение. Явные ожидания (Explicit Waits) — более гибкий и точный механизм. С их помощью можно ожидать наступления конкретного условия (например, видимости элемента) для определённого элемента, используя комбинацию `WebDriverWait` и `ExpectedConditions`, что улучшает синхронизацию тестов со состоянием веб-приложения;

2. Программный метод представляет собой формализованный алгоритм, реализованный в виде кода — последовательности инструкций, необходимых для решения конкретной задачи;

3. Для организации распределённого выполнения тестов существует Selenium Grid. Эта архитектура «сервер-клиент» позволяет запускать скрипты `WebDriver` на удалённых машинах, централизованно управляя браузерами на разных узлах сети.

ВЫВОД

В результате выполнения работы были получены практические навыки по настройке и использованию фреймворка Selenium для автоматизации тестирования веб-интерфейсов.